

VisionInspect AI: Manufacturing Defect Detection & Quality Inspection System

Infosys Springboard Internship 7.0

**Milestone 4:
Individual Contribution Report - Team 1**

Prepared by: **LEELA SOWMYA PAPPU**

Section 1: Executive Summary & Overview

During Milestone 4, the main focus was on connecting the backend inspection results with the frontend and improving the overall usability of the application.

The major contributions included the development of the Owner Dashboard, live inspection information, Quality Certificate generation, manual decision override, backend API fixes, and Landing Page improvements.

The main contributions during this milestone were:

- Development of the Owner Dashboard with charts and inspection statistics.
- Addition of HUD-style cards for live inspection information.
- Fixing issues in the backend analytics APIs.
- Addition of a manual decision override option.
- Development of PDF Quality Certificates.
- Improvement of the Landing Page and navigation.
- Testing of the frontend and backend features.

Section 2: Owner Dashboard & Live Analytics

2.1 Live Inspection Dashboard

The Owner Dashboard was developed to provide a quick view of the inspection results. It displays important information related to the overall quality inspection process.

The dashboard includes:

- ✓ Total number of inspections
- ✓ PASS and FAIL counts
- ✓ Average anomaly score
- ✓ Daily inspection results
- ✓ Defect trends
- ✓ Defect information based on product categories

Recharts was used to display the inspection data in graphical form. Charts were added for defect trends, severity levels, and defect counts for different product categories.

This makes it easier for users to understand the inspection results without having to go through the raw data.

2.2 HUD Telemetry Cards

1. HUD-style metric cards were added for different product categories such as Grid, Hazelnut, and Screw.
2. These cards display the latest inspection information and provide a quick view of the current inspection status.
3. The cards were connected with the backend data so that changes in inspection results are reflected in the frontend. This reduces the need for manually refreshing the page to check updated information.



Section 3: Backend API Fixes

While integrating the dashboard with the backend, some issues were found in the analytics APIs. In some cases, the required data was not being returned correctly to the frontend.

The FastAPI `/analytics/*` routes were checked and the issues related to data handling and API responses were fixed.

The SQLAlchemy queries used for retrieving inspection statistics were also reviewed and modified where required. This helped in getting the required calculations directly from the database.

The queries included information such as:

- Product-wise failure rates
- Defect trends
- Inspection counts
- Severity-related information

After the changes, the API responses were tested to make sure the expected data was being received by the frontend.

```
▼ TERMINAL
INFO: 127.0.0.1:61433 - "GET /api/analytics/defect-trends HTTP/1.1" 200 OK
INFO: 127.0.0.1:62137 - "GET /api/analytics/severity-distribution HTTP/1.1" 200 OK
INFO: 127.0.0.1:64032 - "GET /api/analytics/production-quality HTTP/1.1" 200 OK
WRN-50-2 PatchCore extractor on: cpu (global=3584, patch=1536x196)

===PREDICTION===
{'is_defective': True, 'classification': 'Defective', 'matched_category': 'zipper', 'anomaly_score': 35.256218, 'threshold'
: 21.148432, 'confidence': 0.99, 'distance_ratio': 1.6671, 'is_unknown': False}
=====
Defect Type : Surface Scratch
Size Score : 2.083253860473633
Location: 80.0
Type Score : 60.0
Confidence : 99.0
Overall : 55.42
Decision : FAIL
=====
INFO: 127.0.0.1:64032 - "POST /api/inspect HTTP/1.1" 200 OK
INFO: 127.0.0.1:61433 - "GET /api/analytics/severity-distribution HTTP/1.1" 200 OK
INFO: 127.0.0.1:49675 - "GET /api/analytics/defect-types HTTP/1.1" 200 OK
INFO: 127.0.0.1:62137 - "GET /api/analytics/defect-trends HTTP/1.1" 200 OK
INFO: 127.0.0.1:62163 - "GET /api/analytics/production-quality HTTP/1.1" 200 OK
INFO: 127.0.0.1:64032 - "GET /api/analytics/severity-distribution HTTP/1.1" 200 OK
INFO: 127.0.0.1:62137 - "GET /api/analytics/defect-trends HTTP/1.1" 200 OK
INFO: 127.0.0.1:49675 - "GET /api/analytics/defect-types HTTP/1.1" 200 OK
```

Section 4: Inspection Controls & Quality Certificates

4.1 Manual Decision Override

A manual decision override feature was added to handle cases where the AI-generated result may need to be reviewed by an inspector.

For example, if an item is marked as FAIL by the AI but the inspector finds that the item should be PASS, the final decision can be changed manually.

The override action was connected to the backend so that the updated decision is stored in the database. This also helps in maintaining a record of changes made to the original AI result.

4.2 Quality Certificate Generation

A Quality Certificate generation feature was added for inspected products.

The certificate contains important inspection details, including:

- Product/SKU information
- Inspection date and time
- AI confidence score
- Severity score
- Final inspection result

The certificate is generated in PDF format and can be downloaded for documentation or printing.

This provides a simple way to keep a record of the inspection result after the inspection process is completed.

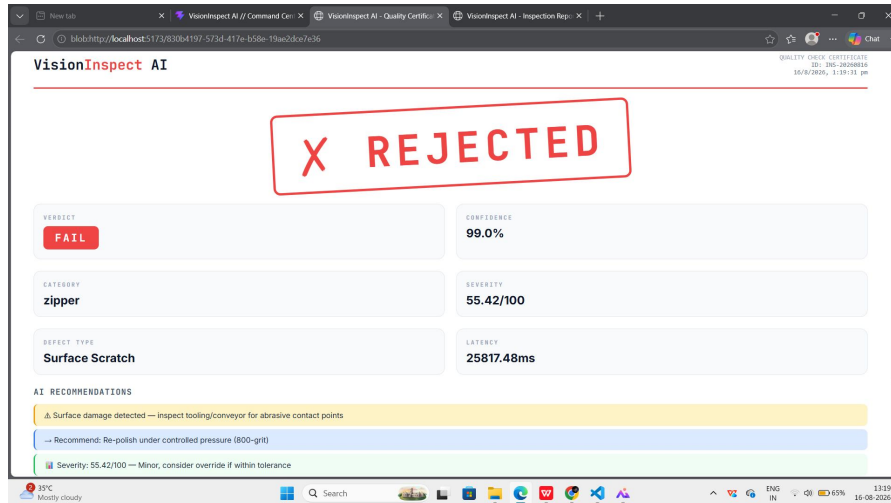


Fig : Quality Certificate

Section 5: Landing Page & UI Improvements

The Landing Page was updated to improve the overall appearance of the application.

Tailwind CSS was used for styling, and some basic animations were added using **Framer Motion**.

The navigation between the main pages was also checked and improved. The main navigation flow includes:

- Landing Page
- Live Inspection Console
- Owner Dashboard

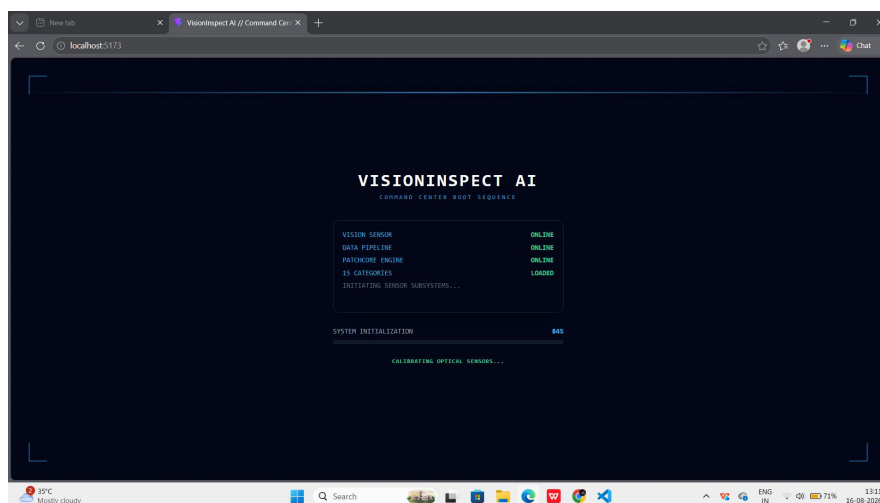


Fig : Enhanced Landing Page : Application Startup

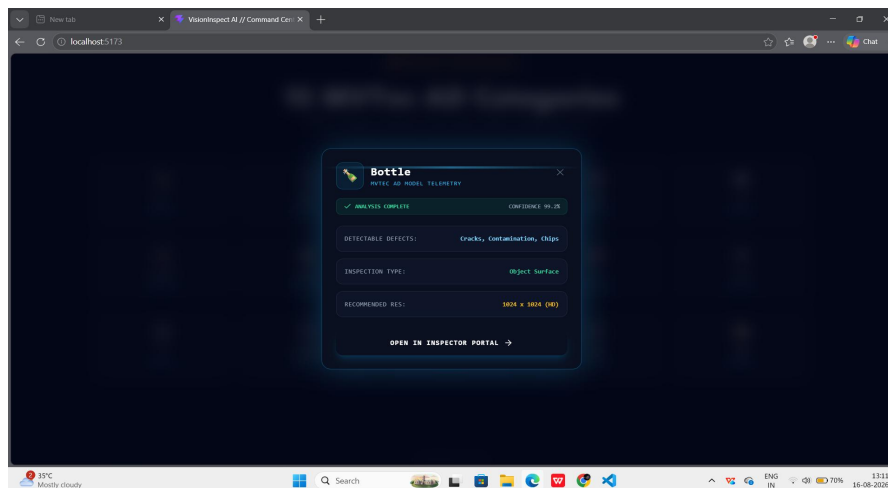
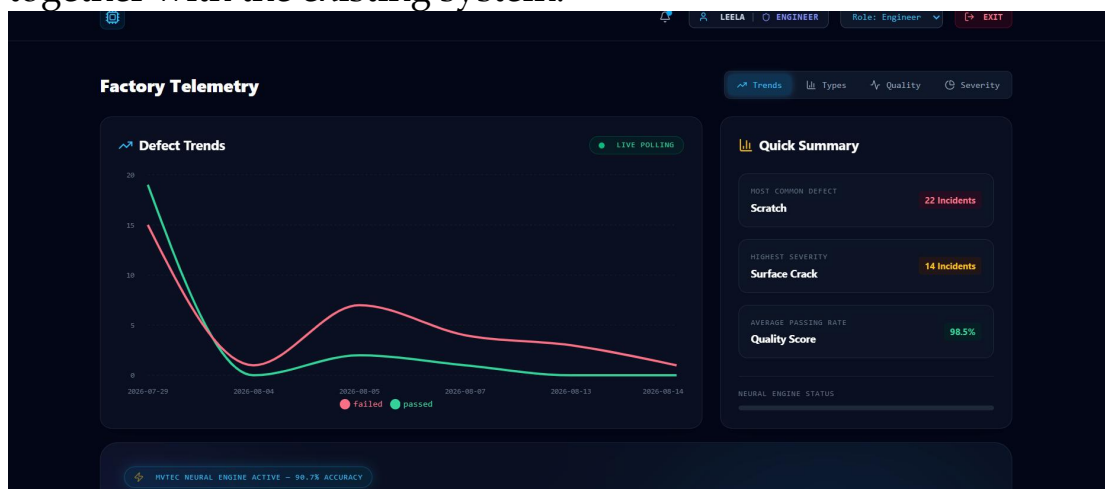


Fig : HUD Telemetry Card

Section 6: Testing & Validation

After completing the development work, the newly added features and modified components were tested individually as well as together with the existing system.



The following areas were tested:

- Analytics API responses
- Owner Dashboard with inspection data
- Chart values and displayed information
- Manual PASS/FAIL decision changes

The backend analytics endpoints were also checked to verify that they were returning the expected data during testing.

Section 7: References

- Python Documentation - Programming language and standard libraries.
<https://docs.python.org/3/>
- OpenCV Documentation - Image processing and computer vision operations.
<https://docs.opencv.org/>
- NumPy Documentation - Numerical computing and array operations used in ML processing.
<https://numpy.org/doc/>
- scikit-learn Documentation - Machine learning utilities and evaluation methods.
<https://scikit-learn.org/stable/>
- FastAPI Documentation - Backend API development and model inference integration.
<https://fastapi.tiangolo.com/>
- React Documentation - Frontend development and user interface components.
<https://react.dev/>